



ZK Voting System

Whitepaper

Parliamentary Governance for DAOs, with Privacy as Phase 2

Version: 1.0
Author: Tyler Delano + Dexter
Status: Phase 1 live on Sepolia
Frontend:
Live Contract:

May 1, 2026

Contents

1	Problem	2
2	Solution: Two-Phase Architecture	2
2.1	Phase 1: Rob’s Rules On-Chain (Now)	2
2.2	Phase 2: ZK Privacy Layer (Future)	2
3	Rob’s Rules Implementation	3
3.1	Parliamentary Features	3
3.2	State Machine	3
3.3	Why Rob’s Rules?	3
4	ZK Circuit Design	3
5	Economic Model	4
6	Deployment	4
6.1	Phase 1 — Live	4
6.2	ZK Layer	4

1 Problem

On-chain DAO voting reveals everything: who voted, what they voted, and when. This violates parliamentary privacy norms and discourages honest voting on controversial motions. Members may vote strategically or avoid voting entirely to stay anonymous.

Current solutions require either:

- Centralized identity providers (ENS, Proof of Humanity), which exclude many legitimate voters.
- Off-chain voting with on-chain settlement, which adds complexity and trust assumptions.

2 Solution: Two-Phase Architecture

2.1 Phase 1: Rob's Rules On-Chain (Now)

Full parliamentary voting using Robert's Rules of Order. Chair-managed voter allowlist on Sepolia. No ZK, no identity requirements. Pure smart contract governance.

Why start here: Get real governance running on-chain before adding complexity. Rob's Rules itself is sophisticated. It handles amendments, reconsiderations, division calls, and quorum-oriented governance behavior that most simple voting systems skip.

2.2 Phase 2: ZK Privacy Layer (Future)

Replace direct with a ZK proof that proves:

1. The voter is on the eligibility allowlist, without revealing which voter.
2. The vote choice is valid (yes, no, or abstain).
3. The nullifier is unique, preventing double-voting.

On-chain result: The contract only sees that a valid proof was cast for a proposal. It does not learn identity, vote content, or a link between voter and vote.

Circuit: already exists, but needs a circom toolchain fix before zkey regeneration.

3 Rob's Rules Implementation

3.1 Parliamentary Features

Feature	Implementation
Motion (proposal creation)	— any eligible voter
Seconding requirement	— another voter must second, reducing spam
Amendments	+ — any voter proposes, chair approves
Debate period	Voting opens after seconding and amendment resolution
Recorded vote (division)	— any member can demand a recorded tally
Reconsideration	— member on the prevailing side can reopen
Majority rule	means passed; abstains do not count

3.2 State Machine

--

3.3 Why Rob's Rules?

Robert's Rules of Order is the standard for deliberative bodies. It handles:

- **Quorum:** minimum participation requirements
- **Seconding:** protection against spam motions
- **Amendments:** constructive modification before voting
- **Division:** recorded vote on demand
- **Reconsideration:** correction of mistakes without a full reproposal

Most blockchain voting systems skip all of this. This system implements the parliamentary process directly.

4 ZK Circuit Design

Circuit: (Phase 2)

Private Inputs

- — prevents double-voting without exposing identity
- — voter identity commitment from the allowlist Merkle tree
- — 0, 1, or 2 for yes, no, or abstain

Public Inputs

- — proves allowlist membership without revealing which voter
- — binds the proof to a specific proposal
- — public record of the spent nullifier

Verification: Groth16 proof verified on-chain by .

5 Economic Model

Phase 1: Free to use. Gas costs are paid by voters. The chair pays for proposal creation.

Phase 2: ZK proof generation is computationally expensive. Planned mitigations:

- Pre-compute proofs client-side, with no server round-trip.
- Use a bouncer or relay for optional gasless submission.
- Deploy on Layer 2, such as Base, for lower gas costs.

6 Deployment

6.1 Phase 1 — Live

- Contract: (Sepolia)
- Frontend:
- Chair:

6.2 ZK Layer

Phase 2 requires circom 2.1.x or Sindri to fix the broken proving toolchain.